Lesson notes · for you

Constraint programming: propagation and search

Optimization · Level 4 · Lesson 3

Time	Materials
25 min	Your phone, or a computer with Colab; A printed 4 × 4 or 9 × 9 sudoku and a pencil

Learning goals

- Describe a problem with domains and constraints, and propagate by hand.
- Explain fail-first search, global constraints and symmetry breaking.
- Model a puzzle in CP-SAT, and read what INFEASIBLE proves.

Activity: Be the solver

1. Propagate (8 min): On a printed sudoku, pencil the candidates into each empty cell, then cross out every digit already in its row, column or box. Place any cell left with one candidate and repeat until nothing changes.
2. Fail-first (7 min): If you are stuck, choose the cell with the fewest candidates, guess, and propagate again. Mark each guess so you can undo it at a dead end.
3. Count (5 min): Compare with a partner who guessed in reading order instead. Who needed fewer guesses, and why?
4. Model (5 min): Write the rules as CP-SAT lines: a variable per cell with domain 1 to 9, all-different for each row, column and box. Run the 4 × 4 version in the code step.

Any sudoku from a newspaper works. A pencil and an eraser make undoing guesses easy.

Discussion

- Why is fail-first a good rule, even though it guesses where you are least sure?
- Which real problems have symmetry worth breaking?
- How would you explain an INFEASIBLE roster to the people who wrote its rules?

Check yourself

- Explain in two sentences why all-different is stronger than its 36 pairwise rules.

Try it this week

Write the 4 × 4 sudoku in CP-SAT, boxes and all. Then remove one given and count the solutions with a solution callback (solver.parameters.enumerate_all_solutions = True). How many givens can you remove before the puzzle has more than one answer?

Worksheet

Constraint programming: propagation and search

Optimization · Level 4 · Lesson 3

Name _____ Class _____ Date _____

1 **Three empty cells: one has two candidates, two have three. Where does fail-first search guess?**

◯ In one of the cells with three ◯ In the first cell in reading order ◯ In the cell with two candidates

2 **Which statements about propagation are true? Choose all that apply.**

☐ It never removes a value that a solution needs ☐ On its own it always solves the problem

☐ All-different can remove values that pairwise ≠ rules miss ☐ It picks the most likely value and moves on

☐ It removes values that can't be part of any solution

3 **This 4 × 4 sudoku model has rows and columns, but the solver's grid breaks the 2 × 2 boxes. Add an all-different constraint for each box inside the loop.**

```
from ortools.sat.python import cp_model
g = [[1, 0, 0, 0], [0, 0, 3, 0], [0, 4, 0, 0], [0, 0, 0, 2]]
m = cp_model.CpModel()
c = [[m.new_int_var(v or 1, v or 4, "") for v in r] for r in g]
for i in range(4):
    m.add_all_different(c[i])
    m.add_all_different([row[i] for row in c])
s = cp_model.CpSolver()
print(s.status_name(s.solve(m)))
print([[s.value(v) for v in r] for r in c])
```

Write your program:

4 **This eight-queens model should stop any two queens sharing a column or a diagonal (q[i] is the column of the queen in row i), but the solver puts all eight on one diagonal. Which line is wrong?**

Circle the line with the mistake. Write it correctly:

```
1  from ortools.sat.python import cp_model
2  n = 8
3  m = cp_model.CpModel()
4  q = [m.new_int_var(0, n - 1, "q%d" % i) for i in range(n)]
5  m.add_all_different(q)
6  m.add_all_different([q[i] + i for i in range(n)])
7  m.add_all_different([q[i] - 1 for i in range(n)])
8  s = cp_model.CpSolver()
9  s.solve(m)
```

5 **If CP-SAT reports INFEASIBLE, giving it more time won't find a solution.**

◯ True ◯ False

✎ **Think of a scheduling problem you know (a timetable, a sports fixture list, a rota). Which constraint would you model as a global constraint, and which symmetry could you break?**

Check your answers

Constraint programming: propagation and search

Optimization · Level 4 · Lesson 3

1 **In the cell with two candidates**

Fail-first chooses the variable with the smallest domain.

2 **It removes values that can't be part of any solution; It never removes a value that a solution needs; All-different can remove values that pairwise ≠ rules miss**

Propagation is safe deduction: it removes only impossible values, so no solution is lost, and global constraints deduce more than their pieces.

3

```
from ortools.sat.python import cp_model
g = [[1, 0, 0, 0], [0, 0, 3, 0], [0, 4, 0, 0], [0, 0, 0, 2]]
m = cp_model.CpModel()
c = [[m.new_int_var(v or 1, v or 4, "") for v in r] for r in g]
for i in range(4):
    m.add_all_different(c[i])
    m.add_all_different([row[i] for row in c])
    m.add_all_different([c[i // 2 * 2 + r][i % 2 * 2 + q]
                         for r in (0, 1) for q in (0, 1)])
s = cp_model.CpSolver()
print(s.status_name(s.solve(m)))
print([[s.value(v) for v in r] for r in c])
```

Givens are fixed through their domains (v or 1, v or 4), and three families of all-different constraints do the rest.

4 **Line 7:** `m.add_all_different([q[i] - i for i in range(n)])`

q[i] − 1 is all-different exactly when q[i] is, so line 7 repeats line 5 and one family of diagonals goes unprotected; q = 0, 1, …, 7 puts every queen on it.

5 **True**

INFEASIBLE means the search proved that no solution exists.